# Nevis: Forward Deployed Engineer Home Task

Candidate:

---

## Context

**Nevis** is an AI platform for wealth management. When a new firm (a Registered Investment Advisor) becomes a customer, their client data lives in whatever systems they happen to use: most often Salesforce, sometimes a CRM improvised in Notion, a custodian's portfolio spreadsheet, a Slack channel full of institutional knowledge. None of it is in our shape. A **Forward Deployed Engineer (FDE)** sits with that firm, understands their business, and gets their book mapped into the Nevis **canonical data model** so the platform can use it.

The hard part is not moving bytes. It is that the mapping is **half mechanical, half judgment**: some fields map cleanly, and a lot of it is ambiguous, requires a business assumption, or can only be resolved by asking the right person the right question. An FDE who asks the client 400 questions is useless; so is one who silently guesses and gets the numbers wrong. The skill is doing as much as can be done confidently and automatically, and flagging the rest as precise, answerable questions.

That judgment gets resolved by talking to the client. Onboarding is an **iterative loop**: you ask the firm's ops people a batch of questions, encode what they tell you, re-run, and come back with a tighter batch about whatever is still unresolved. **You are joining that loop partway through.** Round 1 has already happened, some questions have been asked and answered (that's the Slack thread), and part of your job is to produce round 2: the next batch of questions, for exactly the things round 1 didn't settle.

**This task evaluates how you build a working system that does this mapping:** one that combines data interpretation, hands-on wrangling, and LLM-driven mapping, automates what it can, and turns the rest into clean, client-ready questions. We want to see running code, not a design doc describing code you would write.

## What you're given

In this package (this particular firm is on the messier end of that spectrum, running their CRM in Notion rather than Salesforce):

- `canonical_model.md`: the target model you must map into (five entities, including interaction).
  *Note rule 6: not every CRM field has a first-class home, so deciding what to preserve, normalize, drop, or flag is part of the job.*
- `sources/notion_export/`: the client's CRM, a raw Notion "Markdown & CSV" export

of **two** databases:

  - `Clients*.csv` + its page folder: one row per client. Alongside the core fields it carries **custom fields** the firm added (risk profile, fee schedule, referred-by, tags); these are inconsistent and several have no obvious canonical home. As before, **the page bodies contain notes not in the CSV**, and some matter.
  - `Meetings*.csv` + its page folder: one row per meeting. The Client column is a Notion relation that flattens to the client's page **title** (a person's name), which you must resolve to a household. Meeting pages carry notes too.

  Usual Notion quirks apply throughout: relations appear as title text (not ids), select values are whatever the user typed, and dates export as prose.

- `sources/custodian_positions.xlsx`: account balances from the custodian. This is the only place investment-account-level data lives.
- `sources/advisor_roster.csv`: the firm's advisors.
- `sources/ops_slack_thread.md`: **round 1 of the loop.** An onboarding Slack thread in which the client's Head of Operations (Dana) has already answered the first batch of questions. It carries business rules you will find in no schema; extract them, encode them, apply them. Do not re-ask what she has already answered.

The dataset is deliberately **messy**: it is a realistic small-firm book, not a clean export. Do not expect clean joins, and do not assume the CSV is the whole story.

## What we want you to produce

### 1. A working agentic pipeline (*the core deliverable*)

Code we can run that ingests the sources, does the mapping, and produces the two outputs in section 2. It must actually run end-to-end on the provided sample. We care about the substance (data interpretation, wrangling, and how you put an LLM to work on the judgment-heavy parts), **not** about code cleanliness, which is explicitly not graded. A rough script that runs and reasons well beats a beautiful one that doesn't run.

What "agentic" means here, concretely (we want to see all of these in the code):

- **A real LLM in the loop for the ambiguous work:** normalizing free-text enums, reading the Notion page bodies, resolving entities across sources, judging confidence. Not one giant do-everything prompt, and not a pure rule engine either: show where the model earns its place and where deterministic code is the better tool.
- **A confidence / auto-apply-vs-flag decision** that routes each record to either the clean output or the clarifications list.
- **A knowledge layer.** When a stakeholder answer resolves an ambiguity ("Legacy means the Harborline book"), it is captured and reapplied automatically, for this client, and ideally reusable across future clients. This is central to the role; build it, don't just describe it.
- **Provenance:** each output value traceable to a rule, a model call, or a human answer.
- **Post-mapping validation** that catches plausible-but-wrong results.

Practical notes:

- **Language:** Python. **Agent framework:** your choice (one line on why).
- **Use a real LLM** (any provider, your own key). Commit the outputs your run produced so we can inspect results even without your key. A mock fallback for reproducibility is fine, but the real LLM path must be implemented and demonstrably used.

## 2. The two outputs your pipeline produces

- **`canonical_output`:** the records your system mapped **confidently**, in the canonical schema (any sensible serialization), carrying provenance.
- **clarifications**, round 2: the next message back to Dana. The items your system could not resolve confidently, written as the batch of questions you would send her now. **This is the customer-facing artifact and we read it most closely.** Each item should be a scoped, answerable question, not a shrug: what triggered it, the evidence, candidate options, and your proposed default. It must read as if written by someone who read round 1, so it must **not** re-ask anything the Slack thread already settled, and Dana should be able to answer the whole list in a few lines.

## 3. A short design note + rules write-up *(keep it brief; the code is the point)*

One page maximum. Cover (a) the shape of your system and the key trade-offs you made, and (b) the business rules you extracted from sources/ops_slack_thread.md and how your pipeline encodes them so they apply automatically on the next sync, and get corrected when a rule is wrong or goes stale. Don't over-invest here; a page of prose does not substitute for working code.

## Scope and time

A pipeline that runs end-to-end on the sample and makes sensible, well-evidenced decisions beats a polished design with no working code. **Code cleanliness is explicitly not graded:** we care how you combine data wrangling, AI, and customer judgment. If you run short on time, ship a **narrower pipeline that runs** rather than a broad one that doesn't, and note what you'd tackle next. Document your assumptions and trade-offs.

## Deliverables

A single zip or a GitHub link containing:

1. **Runnable pipeline code** plus a `README.md` with exact setup and run instructions: how to supply an API key and the one command that runs it end-to-end.
2. **The two outputs** (`canonical_output` and clarifications) committed as files, exactly as produced by your run.
3. **A short `DESIGN.md`:** the one page design note and Slack rules write-up.

Submit the deliverables within **5 days** by email.